Software Engineering - Lab 2: CI/CD for a Cross-Platform Application

Objective: This assignment will guide you through building a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline for a cross-platform desktop application using GitHub Actions. You will learn to automate testing, build your application for multiple operating systems, and create automated releases.

Context: We are evolving our simple web calculator into a cross-platform desktop application using Electron.js. You will create a pipeline that automatically tests and builds the application for both Windows and macOS whenever new code is pushed, and automatically creates a new public release when you tag a version.

Prerequisites

1. **Node.js and npm:** You must have Node.js (which includes npm) installed. You can download it from nodejs.org.
2. **Project Files:** Download and extract the calculator-app folder which contains the code for this assignment.

Submission

To complete the assignment, submit the following:

1. The URL to your public GitHub repository: `https://github.com/<your-username>/git-calculator-app-<StudentID>-<YourName>`

2. One ZIP named: `submission_lab2_<StudentID>_<YourName>.zip` with contents:
    1. Local evidence outputs
        1. `test_output_<StudentID>_<YourName>.txt`
        2. `lint_output_<StudentID>_<YourName>.txt`
    2. CI/CD screenshots
        1. `ci_success_build_test_<StudentID>_<YourName>.png` (successful lint + test run)
        2. `ci_matrix_build_artifacts_<StudentID>_<YourName>.png` (3-OS matrix with artifacts)

Part 1: Project Setup for a Desktop App & Sanity Checks

In this part you will (1) create a GitHub repository from the provided **calculator-app** starter, (2) run the app locally, (3) run tests and linting, and (4) export "evidence" text files that capture your local results, then commit everything.

1. **Create your GitHub repository from the starter:** Download & unzip the calculator-app starter to a folder on your machine, then create an empty public GitHub repo with the name: `git-calculator-app-<StudentID>-<YourName>`

   Example: `git-calculator-app-000875-NguyenTienTuanKhiem`

2. **Install Dependencies:** In your local environment, open your terminal in the project root and run this command. It will download all the libraries defined in package.json (like Electron and Jest) into a node_modules folder.
   ```
   npm install
   ```

3. **Run the App Locally:** To confirm everything is working, run the application on your machine.
   ```
   npm start
   ```

   A desktop calculator application window should appear.
4. **Run Tests Locally:** We have included a simple unit test. Run the tests to see the output.
   ```
   npm test
   ```

   The test should pass.

5. **Run Lint Locally:** In programming, lint refers to a static code analysis tool or the process of using such a tool to identify and flag potential issues in source code.
   ```
   npm run lint
   ```

6. **Export "evidence" files (required for submission):** You will capture exact terminal output from tests and lint into two text files to submission

   macOS/Linux (bash/zsh):
   ```
   npm test --silent > test_output_<StudentID>_<YourName>.txt 2>&1
   npm run lint --silent > lint_output_<StudentID>_<YourName>.txt 2>&1
   ```

   Windows PowerShell:
   ```
   npm test --silent *>&1 | Out-File -FilePath test_output_<StudentID>_<YourName>.txt -Encoding utf8
   npm run lint --silent *>&1 | Out-File -FilePath lint_output_<StudentID>_<YourName>.txt -Encoding utf8
   ```

7. **Fix the warnings from the Lint command.**

8. **Commit the New Structure:** Stage and commit all the new and modified files to your main branch.

   If you haven't done so, create your git repository with:

   ```
   git init -b main
   ```

   Then:

   ```
   git remote add origin <YOUR-REMOTE-ORIGIN>
   git add .
   git commit -m "Build: Set up Electron project with testing"
   git push origin main
   ```

Part 2: Continuous Integration with GitHub Actions

Now, we will create a GitHub Actions workflow to automate our tests. This is the "CI" part of CI/CD.

1. **Create Workflow Directory:** In your project's root, create a new directory structure: .github/workflows. Note the . at the beginning of the .github folder.
2. **Create Workflow File:** Inside .github/workflows, create a new file named build-and-test.yml.
3. **Define the Workflow:** Copy the following YAML code into your build-and-test.yml file. This workflow will trigger on every push to the main branch. It defines a single "job" called test that installs dependencies and runs our unit tests.

```yaml
name: Build and Test

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test:
    runs-on: ubuntu-latest # Use a standard Linux environment for testing

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      - name: Lint
        run: npm run lint

      - name: Run unit tests
        run: npm test
```

4. **Commit the Workflow:** Stage and commit the new workflow file.

```
git add .github/workflows/build-and-test.yml
git commit -m "CI: Add workflow for running tests automatically"
git push origin main
```

5. **Verify the Workflow Run:** Go to your GitHub repository and click on the "Actions" tab. You will see your "Build and Test" workflow running. Click on it to see the details of the test

job and verify that all steps completed successfully. Screenshot of a successful CI run showing lint + tests:

`ci_success_build_test_<StudentID>_<YourName>.png`

Part 3: Cross-Platform Builds

Next, let's extend our workflow to build the installable application for both Windows and macOS.

1. **Modify the Workflow:** Update your build-and-test.yml file. We will add a new job called build that depends on the test job succeeding. This job will use a strategy matrix to run twice: once on a Windows runner and once on a macOS runner.
   Replace the entire content of build-and-test.yml with this:

```
name: Build, Test, and Release

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install
      - run: npm run lint
      - run: npm test

  build:
    needs: test # This job will only run if the 'test' job succeeds
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest] # Define the OSes to build
    runs-on: ${{ matrix.os }} # Use the OS from the matrix

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install dependencies
        run: npm install

      - name: Build application
        run: npm run build

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
```

```
name: calculator-${{ matrix.os }} # Name the artifact based on the OS
path: dist/ # The folder where the build output is stored
```

2. **Commit and Push:** Commit and push the updated workflow file. Check the "Actions" tab again. You will now see the build job with three parallel runs for Windows, macOS, and Linux. After they complete, you can find the packaged applications available for download in the "Artifacts" section of the workflow summary page. Screenshots the workflow run showing three parallel build jobs:

   `ci_matrix_build_artifacts_<StudentID>_<YourName>.png`

Grading Rubric

Part	Criteria	Points
1. Local Setup & Evidence	Repo created as `calculator-app-pro-<StudentID>-<YourName>` (public)	10
	`test_output_*.txt` present and contains Jest run output	7.5
	`lint_output_*.txt` present and contains ESLint run output	7.5
	Subtotal	**25**
2. CI: Build & Test Workflow	Workflow file at `.github/workflows/build-and-test.yml`	8
	CI triggers on push/PR to `main`	4
	Steps: install, **lint**, **test** (all succeed)	8
	Screenshot of successful CI run provided	10
	Subtotal	**30**
3. Cross-Platform Builds	Build job depends on tests (`needs: test`)	4
	Matrix over **ubuntu-latest, windows-latest, macos-latest**	8
	`npm run build` produces artifacts on each OS	10
	Artifacts uploaded via `actions/upload-artifact` (screenshot shows artifacts)	8
	Subtotal	**30**
4. Submission Quality Subtotal	Public repo URL correct & accessible	4
	Evidence files & screenshots complete and correctly named	6
	Repository hygiene (clear commits, no secrets, `.gitignore` exists)	5
	Subtotal	**15**
	TOTAL	**100**

Good luck!